

Information Retrieval

Chapter 12: Language Models for Information Retrieval

Study Notes — Sections 12.1.3 onwards

Includes all student Q&A and clarifications

Section 12.1.3 — Multinomial Distributions Over Words

Core Intuition

In a unigram model, word order is completely ignored — only word counts matter. The sentences 'dog bites man' and 'man bites dog' get the exact same probability. Because order doesn't matter, we call this a **bag-of-words model**. The proper probability model for this is the **multinomial distribution**.

The Full Multinomial Formula

$$P(d) = \left[\frac{L_d!}{(tf(t_1, d)! \times tf(t_2, d)! \times \dots \times tf(t_M, d)!)} \right] \times P(t_1)^{tf(t_1, d)} \times P(t_2)^{tf(t_2, d)} \times \dots \times P(t_M)^{tf(t_M, d)}$$

Variable Definitions

Symbol	Meaning
L_d	Total number of tokens in document d
$tf(t_i, d)$	How many times term t_i appears in document d
M	Size of the vocabulary (total unique words)
$P(t_i)$	Probability of word t_i under the language model
The big fraction	Multinomial coefficient — counts orderings of the bag of words

The Two Parts of the Formula

Part	What It Is	Depends on Document?
Part A (Multinomial Coefficient)	Counts how many orderings of the query words exist. For IR: depends only on the QUERY, not the document.	NO → DROP IT
Part B (Word Probabilities)	$P(t_1)^{tf} \times P(t_2)^{tf} \times \dots$ Actual probability product — changes per document.	YES → KEEP IT

Why We DROP the Multinomial Coefficient in IR

When ranking documents for a fixed query, the multinomial coefficient (Part A) is computed from query statistics ($L_q, tf(t,q)$) — these are the same for every document. So it is a constant that cancels out when comparing documents. Dropping it does not change which document ranks higher.

Think of it like: if every student's score is multiplied by the same constant ($\times 2$), the ranking doesn't change. The constant is irrelevant for ranking.

Formula Actually Used in IR

$$P(q \mid M_d) = \text{PRODUCT over } t \text{ in } q \text{ of } P(t \mid M_d)^{tf(t,q)}$$

For short queries where each term appears once ($tf=1$), this simplifies to:

$$P(q \mid M_d) = \text{PRODUCT over } t \text{ in } q \text{ of } P(t \mid M_d)$$

Key Takeaways from 12.1.3

- Unigram model = bag-of-words → word order ignored, only counts matter.
- The proper mathematical model is the multinomial distribution.
- The multinomial coefficient is DROPPED in IR because it's constant for a fixed query.
- What remains (Part B) is the actual scoring formula.
- A document is treated as a 'sample from a topic' — the LM is estimated from it.

■ EXAM TIP: If asked 'why drop the multinomial coefficient?' → say: 'It depends only on the fixed query, so it is identical for all documents and cancels in ranking.'

Student Questions & Clarifications — 12.1.3

Q: The formula shows L_d as 'total tokens in document d' — but doesn't that make the coefficient dependent on the document?

A: Great catch! In the GENERAL multinomial formula, L_d is the document length. But in IR, we use the formula to compute $P(q \mid M_d)$ — the probability of generating the QUERY, not the document. So the coefficient becomes $L_q! / (tf(t_1,q)! \times \dots)$ which depends only on the QUERY — same for every document. My earlier variable table described the general formula; in IR context it is query statistics, not document statistics.

Q: How is dropping the multinomial coefficient related to the general formula?

A: The general formula has two parts. Part A (coefficient) = pure counting number based on the query. Part B = word probability product = changes per document. When ranking, Part A is the same constant K_q for all documents. So: $P(d_1)/P(d_2) = (K_q \times \text{Part_B_}d_1) / (K_q \times \text{Part_B_}d_2) = \text{Part_B_}d_1 / \text{Part_B_}d_2$. K_q cancels. Only Part B matters.

Q: The formula now looks totally different from the general multinomial formula — why?

A: The general multinomial formula is THEORY/BACKGROUND only. The book introduced it to say 'unigram models are really multinomial distributions.' The IR working formula is just Part B after dropping the coefficient. You will NOT use the full multinomial formula in numerical problems — only: $P(q|M_d) = \text{PRODUCT of } P(t|M_d)^{tf(t,q)}$.

Q: In the IR formula, the power $tf(t,q)$ seems to disappear sometimes. Why?

A: For short queries where each term appears only once, $tf(t,q) = 1$, so $P(t|M_d)^1 = P(t|M_d)$. The power is still there — it just equals 1 and becomes invisible. The full correct formula always has the power. Always write it unless told query terms appear only once.

Section 12.2.1 — Using Query Likelihood Language Models in IR

Comparison of IR Approaches

Approach	What It Models
Vector Space (tf-idf)	Similarity between query and document vectors
Probabilistic IR (BIM)	$P(\text{relevant} \mid \text{document, query})$ — directly models relevance
Language Model (LM)	$P(\text{query} \mid \text{document model})$ — how likely is doc to generate query?

The Mathematical Setup — Bayes Rule Derivation

We want to rank documents by $P(d \mid q)$. Using Bayes Rule:

$$P(d \mid q) = P(q \mid d) \times P(d) / P(q)$$

Term	Why Drop It?
$P(q)$	Same for ALL documents → constant → cancels in ranking
$P(d)$	Assumed UNIFORM (all docs equally likely a priori) → constant

What remains:

$$P(d \mid q) \text{ proportional to } P(q \mid M_d) \leftarrow \text{This is the QUERY LIKELIHOOD}$$

The 3-Step LM Retrieval Process

- Step 1: Infer a language model M_d for each document in the collection.
- Step 2: Estimate $P(q \mid M_d)$ — probability of generating the query from each document's model.
- Step 3: Rank documents by these probabilities — highest $P(q \mid M_d)$ wins.

Core Intuition

The user has a prototype document in mind and generates a query based on words that appear in this document. We ask: which actual document in our collection best matches that imagined document?

Key Exam Summary — 12.2.1

Point	Remember
Final ranking formula	$P(d q)$ proportional to $P(q M_d)$
Why drop $P(q)$?	Same for all documents
Why drop $P(d)$?	Assumed uniform across all documents
$P(q M_d)$ meaning	Probability that doc d generates the query
Relevance modeled?	NO — LM avoids explicit relevance modeling

■ EXAM TIP: Common mistake — thinking LM ranks by $P(d|q)$ directly. It simplifies to $P(q|M_d)$ after dropping constants via Bayes Rule.

Section 12.2.2 — Estimating the Query Generation Probability

Overview — Three Stages

- Stage 1: Maximum Likelihood Estimation (MLE) — the basic estimate
- Stage 2: The Zero Probability Problem — what goes wrong with MLE
- Stage 3: Smoothing — the solution

Stage 1: Maximum Likelihood Estimation (MLE)

The simplest way to estimate $P(t \mid M_d)$ is to count how often the term appears in the document and divide by document length.

MLE Formula:

$$P_{\text{mle}}(t \mid M_d) = \text{tf}(t, d) / L_d$$

Symbol	Meaning
$\text{tf}(t, d)$	Raw count of term t in document d
L_d	Total number of tokens in document d

Full MLE Query Probability Formula (Eq. 12.9):

$$P(q \mid M_d) = \text{PRODUCT over } t \text{ in } q \text{ of } \text{tf}(t, d) / L_d$$

MLE Numerical Example

Documents:

- d_1 : 'the dog likes the dog' $\rightarrow L_{d1} = 5$
- d_2 : 'the cat likes fish' $\rightarrow L_{d2} = 4$

Query: 'dog likes'

$$\text{For } d_1: P(q \mid M_{d1}) = (\text{tf}(\text{dog}, d_1) / L_{d1}) \times (\text{tf}(\text{likes}, d_1) / L_{d1}) = (2/5) \times (1/5) = 2/25 = 0.08$$

$$\text{For } d_2: P(q \mid M_{d2}) = (\text{tf}(\text{dog}, d_2) / L_{d2}) \times (\text{tf}(\text{likes}, d_2) / L_{d2}) = (0/4) \times (1/4) = 0 \leftarrow \text{ZERO PROBLEM!}$$

Stage 2: The Zero Probability Problem

If a query term does NOT appear in a document, MLE gives it probability = 0. Since we multiply probabilities, anything $\times 0 = 0$. The entire document gets score ZERO, no matter how relevant it might be for other query terms.

This creates 'conjunctive semantics' — only documents containing ALL query terms get a non-zero score (like AND Boolean search). This is too strict.

■ EXAM TIP: The zero probability problem is also called the SPARSE DATA PROBLEM. It unfairly eliminates potentially relevant documents.

Stage 3: Smoothing — The Solution

Smoothing takes some probability mass away from words that DO appear, and gives it to words that DON'T appear. An unseen word gets a small but non-zero probability — at most the probability it has in the whole collection.

Smoothing Method 1: Linear Interpolation (Jelinek-Mercer)

Formula (Eq. 12.10):

$$P(t \mid d) = \lambda \times P_{\text{mle}}(t \mid M_d) + (1 - \lambda) \times P_{\text{mle}}(t \mid M_c)$$

Symbol	Meaning
λ	Mixing parameter, $0 < \lambda < 1$
$P_{\text{mle}}(t \mid M_d)$	MLE probability of t in document $d = \text{tf}(t,d) / L_d$
$P_{\text{mle}}(t \mid M_c)$	MLE probability of t in the WHOLE COLLECTION $= \text{cf}_t / T$
cf_t	Count of term t across the entire collection
T	Total number of tokens in the entire collection

Intuition of λ :

λ value	Effect
Close to 1	Trust the document more (less smoothing)
Close to 0	Trust the collection more (heavy smoothing)
= 0.5	Equal weight to document and collection

Why it fixes zero probabilities:

$$\text{When } \text{tf}(t,d) = 0 \rightarrow P_{\text{mle}}(t \mid M_d) = 0 \quad P(t \mid d) = \lambda \times 0 + (1 - \lambda) \times (\text{cf}_t / T) = (1 - \lambda) \times (\text{cf}_t / T) \leftarrow \text{NEVER ZERO} \checkmark$$

Smoothing Method 2: Bayesian (Dirichlet) Smoothing

Formula (Eq. 12.11):

$$P(t \mid d) = [\text{tf}(t,d) + \alpha \times P(t \mid M_c)] / [L_d + \alpha]$$

α = smoothing parameter. Large $\alpha \rightarrow$ more smoothing (trust collection more). Small $\alpha \rightarrow$ less smoothing (trust document more). Adds pseudo-counts proportional to collection frequency.

Full Worked Example — Linear Interpolation (Example 12.3)

Collection:

d1: 'Xyzy reports a profit but revenue is down' $\rightarrow$ 8 words

d2: 'Quorus narrows quarter loss but revenue decreases further' $\rightarrow$ 8 words

Total collection = 16 words. $\lambda = 0.5$. Query = 'revenue down'

Step 1: Collection Probabilities

$$P(\text{revenue} \mid M_c) = 2/16 = 1/8 \text{ (appears once in each doc)} \quad P(\text{down} \mid M_c) = 1/16 \text{ (appears once in d1 only)}$$

Step 2: Document MLE Probabilities

d1 (8 words): $P_{\text{mle}}(\text{revenue} \mid M_{d1}) = 1/8$ $P_{\text{mle}}(\text{down} \mid M_{d1}) = 1/8$ d2 (8 words): $P_{\text{mle}}(\text{revenue} \mid M_{d2}) = 1/8$ $P_{\text{mle}}(\text{down} \mid M_{d2}) = 0/8 = 0 \leftarrow \text{'down' not in d2!}$

Step 3: Apply Linear Interpolation (lambda = 0.5)

For d1: $P(\text{revenue} \mid d1) = 0.5 \times (1/8) + 0.5 \times (2/16) = 1/16 + 1/16 = 2/16 = 1/8$ $P(\text{down} \mid d1) = 0.5 \times (1/8) + 0.5 \times (1/16) = 1/16 + 1/32 = 3/32$ $P(q \mid d1) = (1/8) \times (3/32) = 3/256$ For d2: $P(\text{revenue} \mid d2) = 0.5 \times (1/8) + 0.5 \times (2/16) = 1/8$ $P(\text{down} \mid d2) = 0.5 \times 0 + 0.5 \times (1/16) = 1/32$ $P(q \mid d2) = (1/8) \times (1/32) = 1/256$

Step 4: Rank

$P(q \mid d1) = 3/256 > P(q \mid d2) = 1/256$ Final Ranking: $d1 > d2$ ✓ Note: d2 got a NON-ZERO score even though 'down' wasn't in it – that's smoothing working!

The Complete Working Formula (Eq. 12.12)

$P(d \mid q)$ proportional to $P(d) \times \text{PRODUCT over } t \text{ in } q \text{ of: } [(1 - \text{lambda}) \times P(t \mid M_c) + \text{lambda} \times P(t \mid M_d)]$ With $P(d)$ uniform (dropped): $P(d \mid q)$ proportional to $\text{PRODUCT over } t \text{ in } q \text{ of: } [(1 - \text{lambda}) \times P(t \mid M_c) + \text{lambda} \times P(t \mid M_d)]$

Common Student Mistakes

- ✗ Forgetting to smooth — never use raw MLE for ranking, always smooth.
- ✗ Swapping lambda and (1-lambda) — lambda goes with the DOCUMENT, (1-lambda) with COLLECTION.
- ✗ Computing collection probabilities wrong — count term across ALL docs, divide by TOTAL tokens.
- ✗ Forgetting $P(d)$ — in most problems it's uniform so drop it, but mention this assumption.

■ EXAM TIP: Smoothing serves TWO purposes: (1) avoid zero probabilities, (2) implement term weighting similar to IDF — rare collection words that appear in a document get boosted relative to common words.

Student Questions & Clarifications — 12.2.2

Q: So MLE is built on top of the base formula, and we use MLE with smoothing in numericals?

A: Exactly right! Here is the hierarchy: BASE FORMULA: $P(q \mid M_d) = \text{PRODUCT of } P(t \mid M_d)^{\text{tf}(t,q)} \leftarrow$ the goal MLE gives us: $P_{\text{mle}}(t \mid M_d) = \text{tf}(t,d) / L_d \leftarrow$ raw estimate SMOOTHING fixes MLE: $P(t \mid d) = \text{lambda} \times \text{MLE} + (1 - \text{lambda}) \times \text{collection} \leftarrow$ final In any numerical: Step 1: Compute MLE for each query term in each document Step 2: Compute collection probability for each query term Step 3: Apply smoothing formula Step 4: Multiply smoothed probs across all query terms Step 5: Rank documents by highest score

Complete LM Pipeline Summary

MLE gives you the RAW ESTIMATE, smoothing FIXES IT, and the base formula COMBINES THEM to give the final score.

Step	What to Do	Formula
1	Estimate raw doc probability (MLE)	$tf(t, d) / L_d$
2	Compute collection probability	cf_t / T
3	Apply smoothing	$\lambda \times (tf/L) + (1-\lambda) \times (cf/T)$
4	Multiply across all query terms	PRODUCT of smoothed $P(t d)$
5	Rank documents	Highest score = top rank